

Correction — TP n°5 : Fonctions Python

R107 · Fondamentaux de la programmation · IUT Béthune

Exercice 1 — hello.py

Fonction hello(nom) qui affiche Hello [nom]! :

```
def hello(nom): """Affiche Hello [nom]!""" print(f"Hello {nom}!") hello("Ramziatou")
```

Exercice 2 — hello2.py

Valeur par défaut "inconnu(e)" pour le paramètre nom :

```
def hello(nom="inconnu(e)": """Affiche Hello [nom]! (par défaut : inconnu(e))"""  
print(f"Hello {nom}!") hello()
```

Exercice 3 — creation_liste.py

Deux fonctions : saisie utilisateur et liste aléatoire.

```
from random import randint def liste_utilisateur(n=5): """ Retourne une liste de n entiers  
saisis par l'utilisateur. Par défaut n vaut 5. """ liste = [] for i in range(n): val =  
int(input(f"Entier {i+1}/{n} : ")) liste.append(val) return liste def liste_aleatoire(n=5,  
bornemin=0, bornemax=100): """ Retourne une liste de n entiers aléatoires entre bornemin  
et bornemax. Par défaut : n=5, bornemin=0, bornemax=100. """ liste = [] for i in range(n):  
liste.append(randint(bornemin, bornemax)) return liste # Instructions de test l1 =  
liste_utilisateur(4) print(l1) l2 = liste_aleatoire(4) print(l2)
```

Exercice 4 — utilisation_creation_liste.py

Import de liste_aleatoire depuis creation_liste.py :

```
from creation_liste import liste_aleatoire print(liste_aleatoire(5, 1, 20))
```

■ Remarque : les tests de creation_liste.py s'affichent aussi car ils ne sont pas encore dans un if `__name__ == '__main__'`. C'est l'objet de l'exercice suivant.

Exercice 5 — creation_liste2.py

Tests déplacés dans if `__name__ == '__main__'` :

```
from random import randint def liste_utilisateur(n=5): """ Retourne une liste de n entiers  
saisis par l'utilisateur. Par défaut n vaut 5. """ liste = [] for i in range(n): val =  
int(input(f"Entier {i+1}/{n} : ")) liste.append(val) return liste def liste_aleatoire(n=5,  
bornemin=0, bornemax=100): """ Retourne une liste de n entiers aléatoires entre bornemin  
et bornemax. Par défaut : n=5, bornemin=0, bornemax=100. """ liste = [] for i in range(n):  
liste.append(randint(bornemin, bornemax)) return liste if __name__ == "__main__": l1 =  
liste_utilisateur(4) print(l1) l2 = liste_aleatoire(4) print(l2)
```

Exercice 5 (suite) — utilisation_creation_liste2.py

Maintenant les tests de creation_liste2.py ne s'affichent plus lors de l'import :

```
from creation_liste2 import liste_aleatoire print(liste_aleatoire(5, 1, 20))
```

Exercice 6 — ip.py

adresses_ip(classe) génère une IPv4 aléatoire de la classe demandée. classe(adresse) identifie la classe d'une adresse :

```
from random import randint def adresses_ip(classe: str = "A") -> str: """ Retourne une
adresse IPv4 aléatoire de la classe donnée. """ if classe == "A": premier = randint(0, 127)
elif classe == "B": premier = randint(128, 191) elif classe == "C": premier = randint(192,
223) elif classe == "D": premier = randint(224, 239) else: # classe E premier =
randint(240, 255) return (f"{premier}.{randint(0,255)}"
f".{randint(0,255)}.{randint(0,255)}") def classe(adresse: str) -> str: """ Retourne la
classe (A, B, C, D ou E) de l'adresse IPv4 donnée. """ premier = int(adresse.split(".")[0])
if premier <= 127: return "A" elif premier <= 191: return "B" elif premier <= 223: return
"C" elif premier <= 239: return "D" else: return "E" if __name__ == "__main__": adr =
adresses_ip("A") print(f"{adr} est de classe {classe(adr)}")
```

Exercice 7 — cercle.py (test du module math)

Programme fourni dans l'énoncé, rien à modifier :

```
from math import pi rayon = float(input("Rayon (cm) ? ")) perimetre = 2 * pi * rayon aire =
pi * rayon ** 2 print(f"Périmètre = {perimetre:.2f} cm") print(f"Aire = {aire:.2f} cm²")
```

Exercice 8 — figures.py (bonus)

Quatre fonctions d'affichage de formes en étoiles :

```
def carre(n: int = 4) -> None: """Affiche un carré plein d'étoiles de côté n.""" for i in
range(n): print("* " * n) def carre_blanc(n: int = 4) -> None: """Affiche un carré creux
d'étoiles de côté n.""" for i in range(n): if i == 0 or i == n - 1: print("* " * n) else:
print("* " + " " * (n - 2) + "**") def tri_rect(h: int = 4) -> None: """Affiche un triangle
rectangle d'étoiles de hauteur h.""" for i in range(1, h + 1): print("* " * i) def
tri_iso(h: int = 4) -> None: """Affiche un triangle isocèle d'étoiles de hauteur h.""" for
i in range(1, h + 1): espaces = " " * (h - i) etoiles = "* " * i print(espaces + etoiles)
if __name__ == "__main__": carre() print() carre_blanc() print() tri_rect() print()
tri_iso()
```